

Parallélisation du code de calcul d'un problème structuré

Clotilde Auber-Joignant, Antoine Levrat

1 Introduction

L'objectif de ce projet est la parallélisation via la librairie MPI d'un code résolvant l'équation de la chaleur 2D puis, dans un second temps, en 3D. Ce rapport détaille les implémentations séquentielles et parallèles des méthodes de Jacobi et Gauss-Seidel, ainsi que l'étude de leurs performances notamment sur le cluster Cholesky.

La parallélisation d'un code de calcul consiste à dissocier les tâches indépendantes d'un programme afin de les exécuter simultanément sur des cœurs ou des processeurs distincts. Cela permet d'optimiser le temps de calcul, ou encore de pouvoir traiter des problèmes plus grands en répartissant la charge de travail et le stockage en mémoire.

1.1 Problème continu

On cherche à résoudre l'équation de la chaleur sur un domaine $\Omega =]0, a[\times]0, b[$. L'équation est donnée par :

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x, y) \quad \text{pour } (x, y) \in \Omega$$

Avec $f(x, y) = 0$. Les conditions aux limites sont :

$$\begin{aligned} u(x, 0) &= U_0 \\ u(x, b) &= U_0 \\ u(a, y) &= U_0 \\ u(0, y) &= U_0(1 + \alpha V(y)), \quad \text{avec } V(y) = 1 - \cos(2\pi y/b) \end{aligned}$$

1.2 Problème discrétisé

La discrétisation par différences finies sur une grille régulière $u_{i,j} = u(i\Delta x, j\Delta y)$ mène au schéma suivant pour $i = 1 \dots N_x$ et $j = 1 \dots N_y$:

$$\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{\Delta x^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{\Delta y^2} = f_{i,j}$$

Le problème est résolu itérativement avec un champ initial $u = U_0$.

2 Partie 1 : Implémentations et validations de code séquentiels

2.1 Méthode de Jacobi

Dans un premier temps, nous avons cherché à implémenter le schéma des différences finies avec la méthode de Jacobi. Le code séquentiel a été construit puis testé en vérifiant la convergence numérique du schéma. Pour cela, nous avons fixé $a = b = 25$, $U_0 = 1$, $\alpha = 0.5$ et pris un maillage en $(N_x + 2) \times (N_x + 2)$ points pour $N_x = 128$. Nous avons fixé le nombre maximal d'itérations à 100 000. Grâce à ce premier test, nous pouvons visualiser la forme de la solution de l'équation à résoudre.[Figure 1].

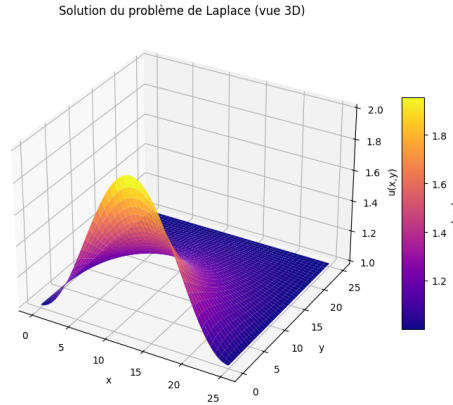


Figure 1 – solution u_{num} sur $]0, 25[\times]0, 25[$ pour $\alpha = 0.5$ et $N_x=N_y=128$

Cependant, cette solution, ne permet pas facilement de déterminer correctement la convergence de l'erreur de discrétisation (estimée par $\|u_{num} - u_{exact}\|_{L^2}$) en fonction du pas de maillage. Pour cette raison, pour la suite de tout nos prochains tests d'étude de convergence, nous choisissons d'utiliser le problème initial avec :

$$u_{exact}(x, y) = \sin\left(\frac{\pi x}{a}\right) \sin\left(\frac{\pi y}{b}\right)$$

Cette solution respecte une condition aux limites $U_0 = 0$ sur l'ensemble du bord. En injectant u_{exact} dans l'équation de la chaleur, nous obtenons le terme source $f(x, y)$ correspondant :

$$f(x, y) = -\pi^2 \left(\frac{1}{a^2} + \frac{1}{b^2} \right) \sin\left(\frac{\pi x}{a}\right) \sin\left(\frac{\pi y}{b}\right)$$

Nous trouvons donc une solution représentée en [Figure 2]

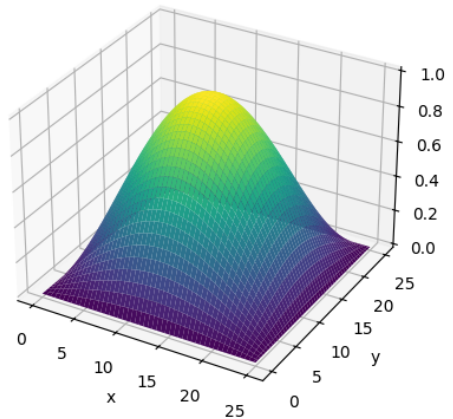


Figure 2 – solution u_{num} sur $]0, 25[\times]0, 25[$ pour $U_0 = 0$ et $N_x=N_y=128$

2.2 Méthode de Gauss-Seidel

Dans un second temps, nous avons cherché à implémenter le schéma des différences finies avec la méthode de Gauss-Seidel. En anticipation d'une mise en place parallèle de cette méthode, nous avons opté pour une coloration rouge-noir des nœuds : À l'itération l , comme nous avons besoin des voisins du point calculé, nous divisons la grille en deux sous-ensembles de sorte que les nœuds rouges ne dépendent que des nœuds noirs, et inversement. De cette manière, nous pouvons calculer tous les rouges d'un coup, puis tous les noirs.

Cette méthode permet de résoudre le problème de l'ordre des calculs qui empêche la parallélisation de Gauss-Seidel.

2.3 Convergence des méthodes

Après avoir confirmé la bonne mise en place des codes séquentiels, nous avons voulu représenter leur convergences : Dans la Figure 4 la vitesse de convergence de chacune des deux méthodes étudiées, en affichant l'erreur obtenue (en norme infinie) en fonction du nombre d'itérations.

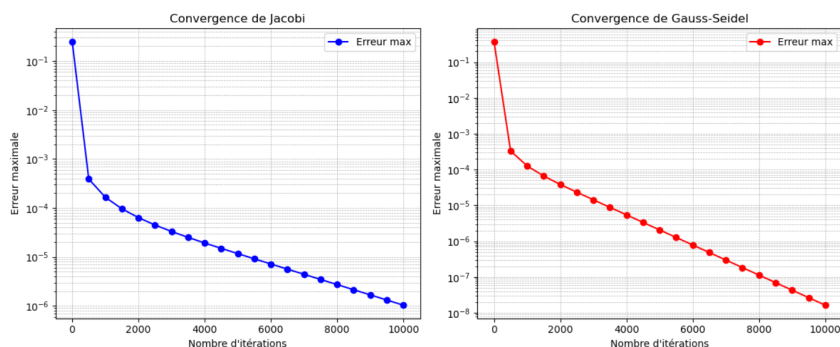


Figure 3 – Comparaison de la convergence pour la méthode de Jacobi (à gauche) et la méthode de Gauss-Seidel (à droite), $N_x = 100$

Nous pouvons voir que, pour un maillage identique, la méthode de Gauss-Seidel offre une meilleure convergence que la méthode de Jacobi. En effet, au cours de nos tests, nous

avons pu constater qu'il ne fallait que 3373 itérations à la méthode de Gauss-Seidel pour obtenir une erreur $< 10^{-5}$, tandis que la méthode de Jacobi nécessitait 5319 itérations pour obtenir une telle précision.

La méthode de Gauss-Seidel s'avère donc plus efficace pour traiter des problèmes plus complexes ou de taille plus importante.

2.3.1 Remarque

Nous avons cherché à déterminer la convergence spatiale et la convergence itérative. Nos premiers tests n'ont pas bien été interprétés. En effet, si le maillage est trop fin ou les itérations trop faibles, la convergence spatiale (en $O(h)$) ne représente pas fidèlement la convergence de la méthode. En [Figure 4] nous observons donc que la courbe, correcte au début, finit par croître, témoignant d'un manque d'itérations.

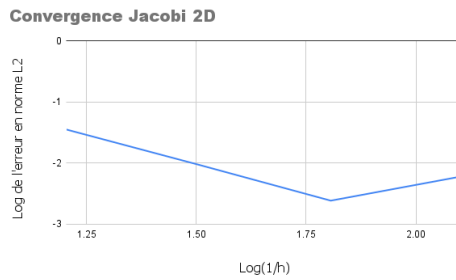


Figure 4 – Convergence Spatiale de Jacobi 2D pour un nombre d'itérations trop faible

Comparaison des convergences spatiales de Jacobi et Gauss-Seidel

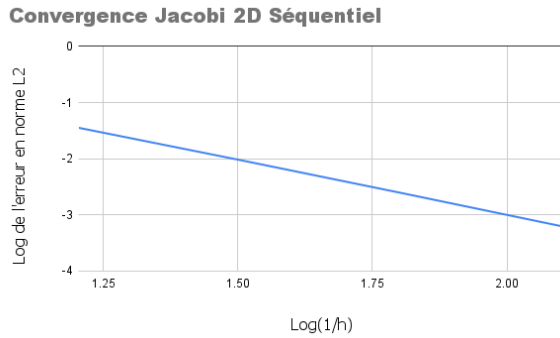


Figure 5 – Convergence spatiale de Jacobi 2D

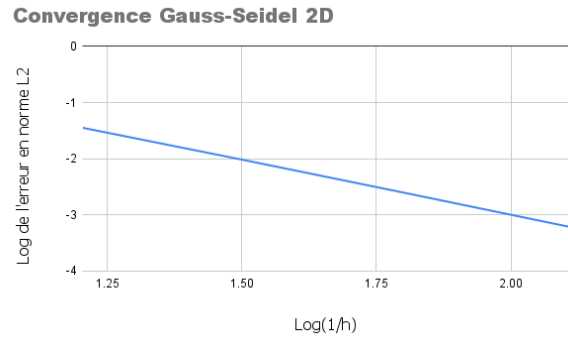


Figure 6 – Convergence spatiale de Gauss-Seidel 2D

Ainsi, après mise au point des itérations nous pouvons observer que sur la [Figure 5] et la [Figure 6], l'erreur décroît linéairement avec une pente de 2. Ce résultat est attendu pour Gauss-Seidel, en effet c'est un schéma d'ordre 2. Cependant, Jacobi est une méthode d'ordre 1, nous devrions avoir une pente de 1. Cette observation récurrente dans nos prochains test est sûrement due à la solution exacte choisie, potentiellement trop régulière.

3 Partie 2 : Implémentation et validation d'un Jacobi parallèle

3.1 Jacobi 2D parallèle

Nous avons parallélisé le code de Jacobi afin de pouvoir utiliser plusieurs processeurs pour un même calcul. Le code a été adapté de manière indépendante du nombre de processeurs afin de prioriser la scalabilité du programme. Nous avons commencé par utiliser des communications bloquantes, ce qui a résulté en un temps de calcul particulièrement long. Nous avons donc fait évoluer le code vers des communications non bloquantes. Pour cela, nous avons modifié la structure du code pour effectuer d'abord les communications, calculer les points qui ne dépendaient pas des données reçues et envoyées (pour recouvrir le temps de communication par du calcul), puis nous nous sommes assurés que les communications étaient terminées avec la commande `MPI_Waitall()` pour effectuer le calcul des points qui en dépendaient.

Nous avons validé le code parallèle en le faisant tourner sur n processeurs, $n = 1, 2, 4, 6, 8, 12, 16$ en vérifiant que l'erreur obtenue correspondait à celle obtenue avec le code séquentiel. On obtient ainsi la [Figure 7] et la [Figure 8]

Convergence spatiale et itérative de Jacobi 2D parallélisé

Convergence Jacobi 2D Parallélisé

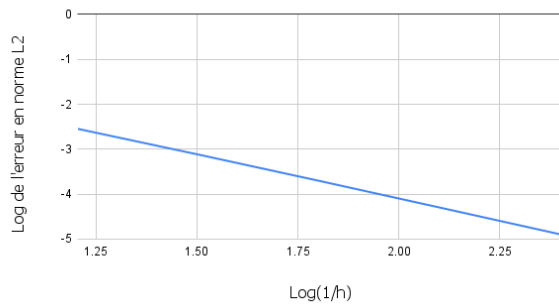


Figure 7 – Convergence itérative

Convergence pour 110000 itérations Jacobi 2D

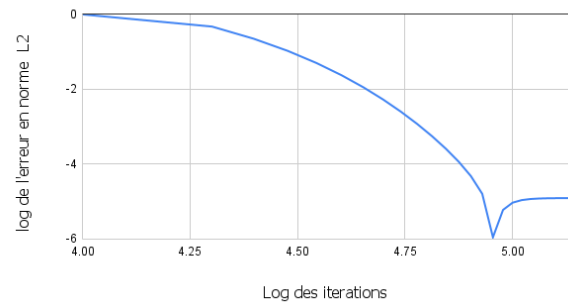


Figure 8 – Convergence spatiale

Ces résultats sont cohérents avec les précédents et montrent encore que la méthode de Jacobi converge en $O(h^2)$ sur cette configuration. Aussi, la [Figure 8] montre que pour un maillage de taille 256×256 et pour un grand nombre d'itérations on a bien une convergence de la méthode.

Nous avons ensuite souhaité étudier la performance parallèle du code en calculant le speedup et l'efficiency. Pour cela, nous avons chronométré la partie itérative du code. Cependant, nous n'obtenions pas les mesures attendues : le temps ne décroissait pas lorsque le nombre de processeurs augmentait.

Nous avons donc décidé de montrer la scalabilité sur nos propres machines, n'ayant pas réussi à obtenir de mesures satisfaisantes sur les machines de l'ENSTA. Nous avons donc augmenté le nombre maximal d'itérations et la taille du maillage, jusqu'à étudier un maillage de taille $N_x = 16000$ pour $N_y = 1000$

Scalabilité Faible et Forte de Jacobi 2D mesurée sur un processeur intel i5 11400H 2.70 GHz

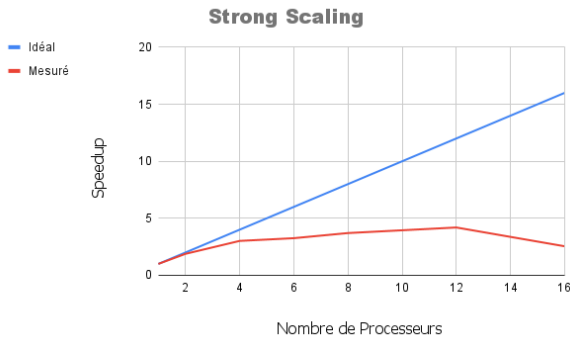


Figure 9 – Scalabilité Forte sur i5

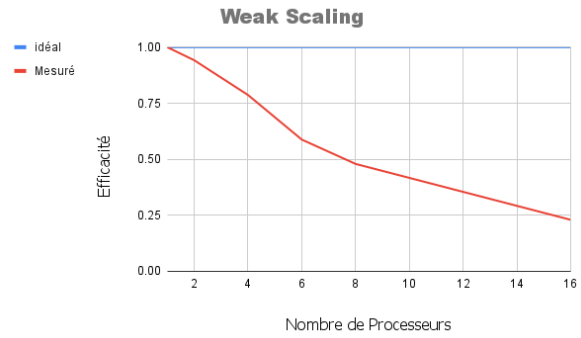


Figure 10 – Scalabilité Faible sur i5

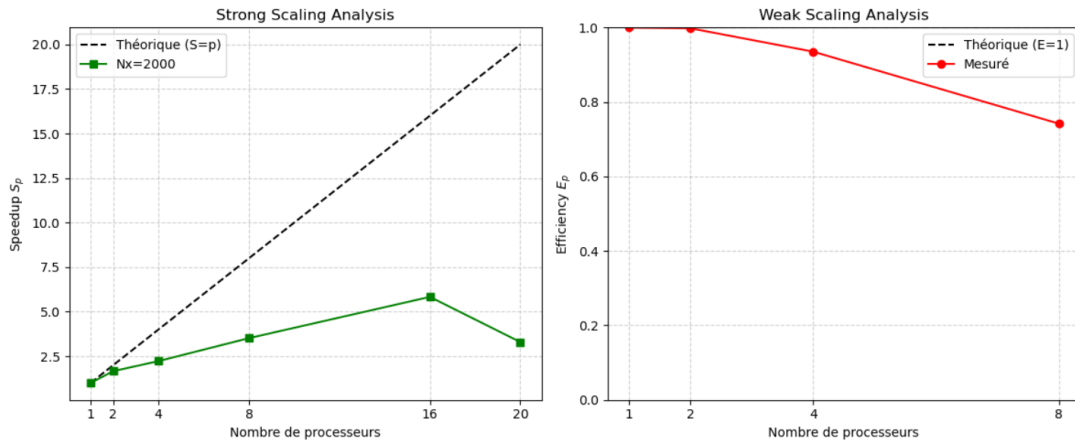


Figure 11 – Scalabilité Faible et Forte mesurée sur 1,2,4,8,16,20 processeurs, sur Cholesky

La scalabilité mesure la capacité d'un algorithme parallèle à exploiter efficacement un nombre croissant de processeurs. On distingue la scalabilité faible, où la taille du problème augmente proportionnellement avec le nombre de processeurs, et la scalabilité forte, où le problème reste de taille fixe.

Remarque : le processeur intel i5 étudié possède 6 cœurs physique mais 12 processeurs logiques, rendu possible par la technologie hyperthread d'intel

On observe en [Figure 9] que par exemple pour 4 processeurs, la vitesse de calcul n'est divisée que par 3 au lieu de 4, en raison des surcoûts liés à la communication et à la synchronisation. De plus, au-delà de 12 processeurs, la vitesse diminue car l'ordinateur n'a plus suffisamment de cœurs physiques disponibles et doit partager les cœurs restants, par ailleurs 2 processeurs qui essaient d'accéder à la même unité mémoire simultanément implique des temps de calculs plus long.

Comme représenté en [Figure 10], les résultats montrent un décrochage assez net. Entre un problème de taille 1 sur un processeur et un problème de taille 6 sur 6 processeurs on perd déjà 41/100 d'efficacité. Cette perte est due notamment aux communications entre processeurs qui devient plus coûteuse. De même on peut visualiser l'hyperthreading intel, de 0 à 6 processeurs on a une pente constante, et par la suite on a aussi une pente constante mais de coefficient différent (plus petit).

La [Figure 11] montre nos tests réalisés sur Cholesky. Le Cluster offre des performances accrues notamment au niveau de la vitesse des communications. On observe ainsi sur la scalabilité faible des mesures bien supérieures au processeur Intel : pour 8 processeurs on était en dessous de 0.50 tandis qu'avec Cholesky on est au dessus de 0.70. De plus les machines utilisées ont 16 cœurs, ce qui se voit très bien sur la courbe de scalabilité forte : on observe un décrochage du speedup au delà de 16 processeurs.

Remarque : Ces mesures montrent que l'exécution sur un nœud de Cholesky présente plusieurs avantages. Cependant, en termes de speedup, les performances pour 12 processeurs restent quasi similaires entre les deux machines ($Sp \approx 4.5$).

3.2 Jacobi 3D parallèle

Nous décidons enfin contre toute attente de coder la méthode de Jacobi en 3D et parallélisée en une dimension. En effet, le code en 2 dimensions utilisait déjà le principe de stocker la matrice de u sous la forme d'un vecteur 1D. Ce passage a permis de faciliter les communications MPI, et ne demande que la création d'une fonction pour passer des indices de la matrice à ceux du vecteur. Passer en 3D découle donc directement du code 2D. On applique ainsi le schéma de différences finies adapté suivant :

$$\frac{u_{i+1,j,k} - 2u_{i,j,k} + u_{i-1,j,k}}{\Delta x^2} + \frac{u_{i,j+1,k} - 2u_{i,j,k} + u_{i,j-1,k}}{\Delta y^2} + \frac{u_{i,j,k+1} - 2u_{i,j,k} + u_{i,j,k-1}}{\Delta z^2} = f_{i,j,k},$$

$$\text{pour } i = 1, \dots, N_x, \quad j = 1, \dots, N_y, \quad k = 1, \dots, N_z.$$

Vis-à-vis des communications, la logique reste la même hormis, pour cette version on envoie et reçoit des plans de taille $(N_y + 2) \times (N_z + 2)$. Enfin nous avons adapté la solution de référence et le terme source tel que :

$$u_{\text{exact}}(x, y, z) = \sin\left(\frac{\pi x}{a}\right) \sin\left(\frac{\pi y}{b}\right) \sin\left(\frac{\pi z}{c}\right)$$

$$f(x, y, z) = -\pi^2 \left(\frac{1}{a^2} + \frac{1}{b^2} + \frac{1}{c^2} \right) \sin\left(\frac{\pi x}{a}\right) \sin\left(\frac{\pi y}{b}\right) \sin\left(\frac{\pi z}{c}\right)$$

On peut alors étudier la convergence de ce code :

Convergence spatiale, itérative et temps d'exécution de Jacobi 3D parallélisé

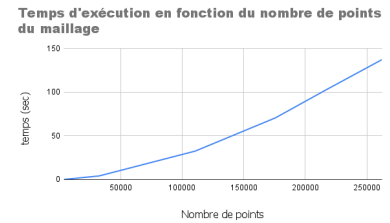
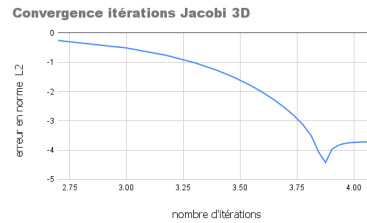
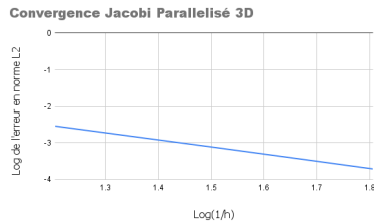


Figure 12 – Convergence spatiale

Figure 13 – Convergence itérative

Figure 14 – Temps d'exécution par itérations

On observe en [Figure 12] que la méthode a toujours une pente de ≈ 2 . Par ailleurs nous avons pris la liberté d'étudier le temps d'exécution en fonction du nombre de points de maillage. Aussi, on constate en [Figure 14] que le temps de calcul augmente drastiquement quand on raffine le maillage de $N_x = N_y = N_z = 32$ à 64 par exemple. Par acquis de conscience nous avons aussi refait les courbes de Scalabilité pour ce programme 3D. On obtient sans surprise en [Figure 15] et [Figure 16] des résultats très similaires à ceux trouvés précédemment.

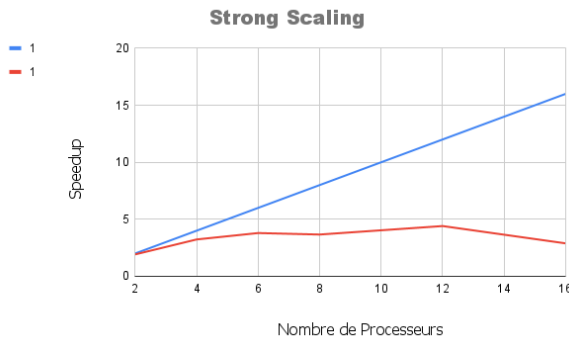


Figure 15 – Scalabilité Forte sur i5

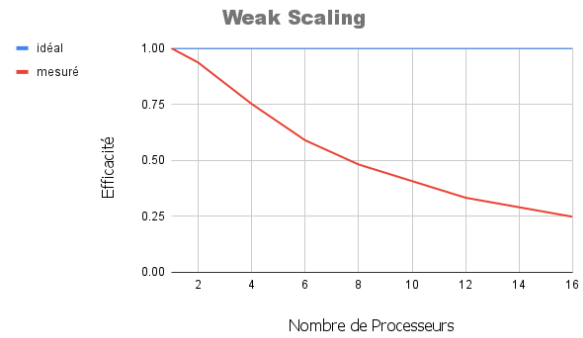


Figure 16 – Scalabilité Faible sur i5

4 Conclusion :

Ce projet nous a permis d'aborder de manière concrète la parallélisation d'un code de calcul scientifique, depuis l'implémentation séquentielle jusqu'à la version parallèle tridimensionnelle. Après avoir validé les méthodes de Jacobi et de Gauss-Seidel en 2D, nous avons confirmé numériquement la convergence des schémas, avec un ordre spatial proche de 2 pour les deux méthodes. Ces validations ont constitué une base solide pour la mise en œuvre parallèle.

La parallélisation du code de Jacobi, d'abord en 2D puis en 3D, a nécessité une réflexion sur la gestion des communications. L'utilisation de communications non bloquantes et le stockage de la solution sous la forme d'un vecteur 1D a permis d'améliorer significativement les performances. L'étude de scalabilité, menée à la fois sur une machine personnelle et sur le cluster Cholesky, a mis en évidence les limites liées au coût des échanges entre processeurs ainsi qu'à l'hyperthreading. Les résultats obtenus montrent néanmoins une bonne efficacité jusqu'à un nombre de cœurs correspondant aux ressources physiques disponibles.

Ce travail met ainsi en évidence l'utilité de la parallélisation pour des calculs de grande taille. Une suite naturelle à ce projet serait d'étudier la parallélisation bidimensionnelle du maillage ou de comparer cette méthode à la méthode de Gauss-Seidel parallèle en damier.